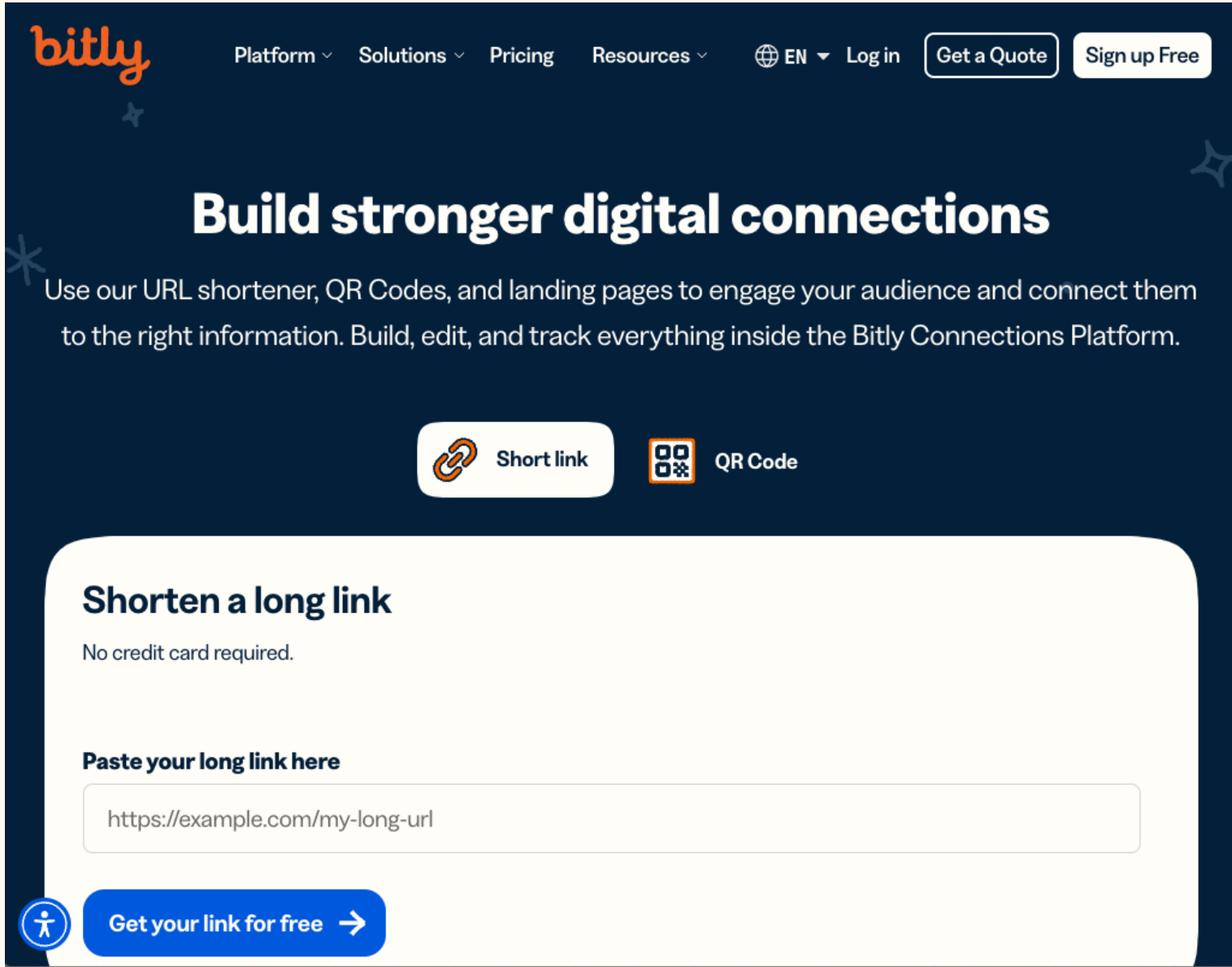


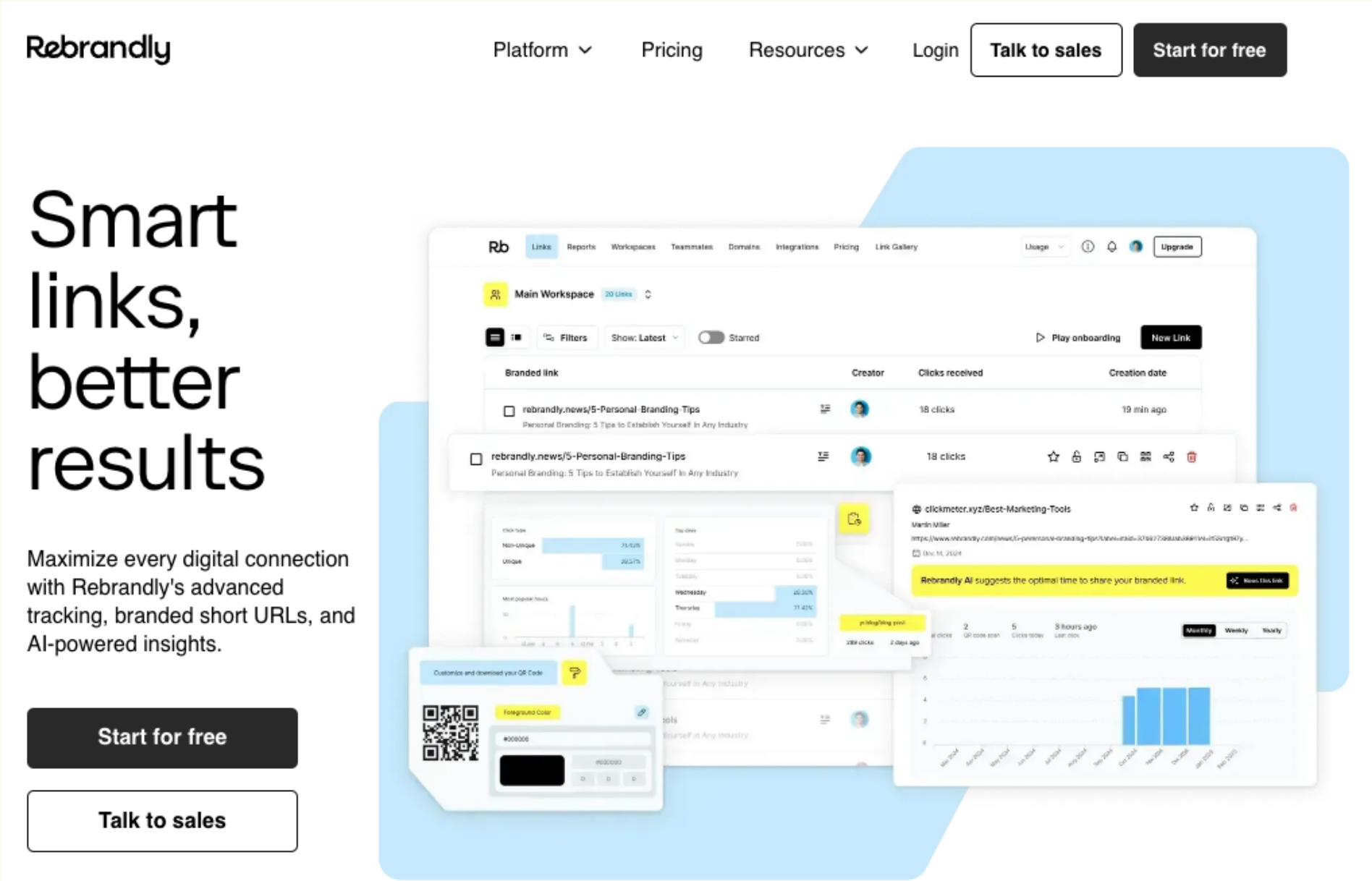
URL Shortener

I. Lesson

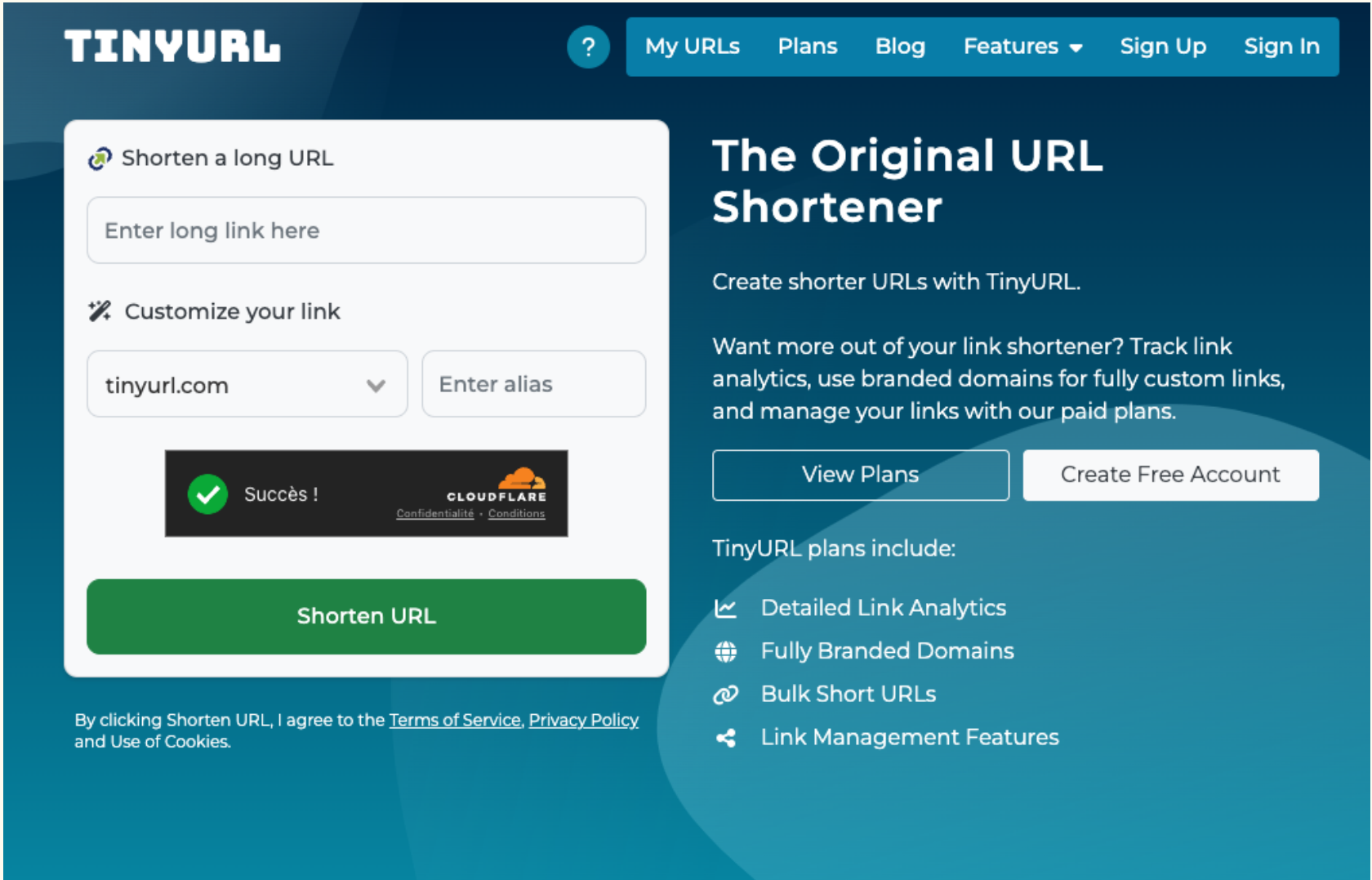
bitly.com



rebrandly.com



tinyurl.com



What are the benefits of using Short urls? 🤔

1. **Easier to share** – Short links are simpler to type, paste, or say out loud.
2. **Cleaner look** – They look tidier in messages, emails, and social media posts.
3. **Save space** – Useful where character limits matter (like on X/Twitter).
4. **Track clicks** – Many shorteners provide analytics to see how many people visited.
5. **Branding** – You can create custom short links that include your brand name.
6. **Hide long query strings** – Makes links less intimidating and more user-friendly.

II. Project

Migrations: Create table

```
CREATE TABLE IF NOT EXISTS urls (  
  id          SERIAL          PRIMARY KEY,  
  original_url TEXT          NOT NULL,  
  short_code  VARCHAR(8)     NOT NULL UNIQUE,  
  click_count INTEGER        NOT NULL DEFAULT 0,  
  created_at  TIMESTAMPTZ    NOT NULL DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ    NOT NULL DEFAULT NOW()  
);
```


Migrations: Insert data

```
INSERT INTO urls (original_url, short_code, click_count) VALUES  
('https://www.google.com', 'ggl123', 15),  
('https://www.github.com', 'gh456', 8),  
('https://www.rust-lang.org', 'rust01', 42)
```

Migrations: Create an index

```
-- Create index for faster lookups by short_code
▷ Run
CREATE INDEX IF NOT EXISTS idx_urls_short_code ON urls(short_code);
```

Benefits of using
index 

Without Index (Table Scan):

- Database reads row 1: "ggl123" ❌ not what we want
- Database reads row 2: "gh456" ❌ not what we want
- Database reads row 3: "so789" ❌ not what we want
- ...continues until it finds "rust01" ✅
- Must check every single row until found

With Index (B-Tree Structure):

The index creates a sorted tree structure like:

```
      "ggl123"
      /    \
    "docs02" "so789"
    /  \    /  \
"crate3" "gh456" "rust01" "yt555"
```

To find "rust01":

1. Start at root: "ggl123" – we want "rust01" (alphabetically after), go right
2. At "so789" – we want "rust01" (alphabetically before), go left
3. Found "rust01" ✅

Routes

```
pub fn url_routes() -> Router<AppState> {  
    Router::new() Router<AppState>  
        .route(path: "/shorten", method_router: post(handler: Url::create_short_url)) Router<AppState>  
        .route(path:("/{short_code}", method_router: get(handler: Url::redirect_to_url)) Router<AppState>  
        .route(path: "/stats/{short_code}", method_router: get(handler: Url::get_url_stats))  
}
```

URL Shortener

Your turn!